Linked Lists

The Presentation

Ruhan Alpaydin

Faculty of Computer and Information Sciences
Yeditepe University

v2026-11-07

Outline

1. Why a linked list?
2. The Link Class
3. The LinkList Class
4. FirstLastList
5. Doubly Linked
6. Uses and Wrap-up
7. The Two Exercises from Previous Years
8. Classic Problems

Why a linked list?

Two ways to store a sequence

Array

`a[0]`, `a[1]`, . . . , `a[n-1]` in one block.
Jump to `a[k]` in $O(1)$.
Insert or delete in the middle: move $n - k$ elements.

Linked list

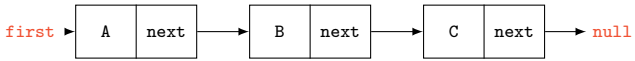
Each element lives in its own **node**, with a `next` reference.
Jump to position k: walk from the head ($O(k)$).
Insert or delete: change two references.

Neither is better: an array is better when you know the position; a linked list is better when you know the node.

What a node is

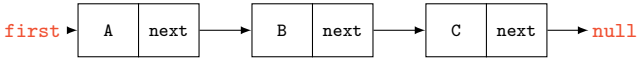
Field	Meaning	Java
data	the value being stored	`int iData`
next	a reference to the next node, or `null`	`Link next`

A linked list is a chain of such nodes (links). The list holds a single reference, `first`, to the head of the chain. The last node in the chain has `next == null`.



This list has three nodes: A, B, C, then `null`.

Structure of a singly linked list



- One reference (`first`) is enough to find every node.
- The list has no `last` unless you add one.
- Every node is reached through the previous node's `next`.

You can only walk forward

Sequential access

To reach position k, start at `first` and follow `next` k times. There is no `a[k]` in a linked list.

- Iteration is the normal way to visit every element.
- Any algorithm that jumps by index is the wrong fit.
- The JDK's `LinkedList.get(i)` is $O(i)$, which is why you use it as a `Deque`, not as a `List`.

Insert at the head

1. Create the new node.
2. Point its `next` to the old `first`.
3. Point `first` at the new node.

Two references

The rest of the list is untouched. $O(1)$.

Delete at the head

1. Save the old `first`.
2. Advance `first` to `first.next`.
3. Return the saved node's data.

One reference

The removed node becomes unreachable; the GC reclaims it. $O(1)$.

Insert at the tail

- Without a `last` reference: walk from `first` to the end, then attach. $O(n)$.
- With a `last` reference: attach at `last`, then move `last`. $O(1)$.

Double-ended list

`FirstLastList` keeps both `first` and `last`. That is the whole difference between a stack (only needs `first`) and a queue (needs both).

The Link Class

Link: three fields

codeoutput/Link.java — Java

```
1  public class Link {
2
3      public int iData;
4      public double dData;
5      public Link next;
6
7      public Link(int id, double dd) {
8      iData = id;
9      dData = dd;
10     }
11
12     public void displayLink() {
13         System.out.print("[" + iData + ", " +
        dData + "] ");
14     }
15 }
```

- `iData`, `dData`: the payload.
- `next`: the reference to the next node, or `null`.
- The constructor leaves `next` at `null` by default.
- `displayLink` prints one node as `[iData, dData]`.

The LinkList Class

LinkList: fields and constructor

codeoutput/LinkList.java — Java

```java
public class LinkList {
    private Link first;

    public LinkList() { first = null; }
```

- One field, `private Link first`: the whole list.
- The constructor sets `first = null`: an empty list.
- No array, no capacity, no size counter.
- The list's state is exactly one reference.

LinkList: insertFirst

codeoutput/LinkList.java — Java

```
5     public void insertFirst(int id, double dd)
       {
6        Link newLink = new Link(id, dd);
7        newLink.next = first;
8        first = newLink;
9     }
```

Two references change

1. the new node's `next` ← the old `first`;
2. `first` ← the new node.

Cost

Two assignments, no loop. $O(1)$.

LinkList: deleteFirst

codeoutput/LinkList.java — Java

```java
    public Link deleteFirst() {
        if (first == null) return null;
        Link temp = first;
        first = first.next;
        return temp;
    }
```

Three statements

1. save the old `first`;
2. advance `first` to `first.next`;
3. return the saved node.

Edge case

Empty list: return `null`, do not crash. $O(1)$.

LinkList: displayList

codeoutput/LinkList.java — Java

```
16      public void displayList() {
17          System.out.print("List (first-->last):
          ");
18          for (Link current = first; current !=
          null; current = current.next)
19              current.displayLink();
20          System.out.println();
21      }
```

- A `for` loop starting at `first`, following `next` until `null`.
- The condition `current != null` stops the walk at the end of the chain.
- Every traversal of a linked list has this shape.

LinkList: find

codeoutput/LinkList.java — Java

```
22      public Link find(int key) {
23          for (Link current = first; current !=
         null; current = current.next)
24              if (current.iData == key) return
         current;
25          return null;
26      }
```

- Same loop shape as `displayList`.
- Inside the loop, test the key: if it matches, return the node.
- If the loop ends, the key is not in the list: return `null`.
- $O(n)$: the only search is a walk.

LinkList: delete

codeoutput/LinkList.java — Java

```
27      public Link delete(int key) {
28          Link current = first, previous = null;
29          while (current != null && current.iData
          != key) {
30              previous = current;
31              current = current.next;
32          }
33          if (current == null) return null;
34          if (previous == null) first = first.
         next;
35          else previous.next = current.next;
36          return current;
37      }
```

The two-pointer walk

`current` walks the chain; `previous` trails one node behind. When `current.iData == key`, you are ready to unlink.

Two cases

- `previous == null`: the match is at the head, so `first` moves.
- `previous != null`: `previous.next` skips over `current`.

LinkList: main() and its output

codeoutput/LinkList.java — Java

```
38      public static void main(String[] args) {
39          LinkList list = new LinkList();
40          list.insertFirst(1, 1.0);
41          list.insertFirst(2, 2.0);
42          list.insertFirst(3, 3.0);
43          list.displayList();
44          System.out.println("find 2 -> " + (list.find(2) != null));
45          System.out.println("find 9 -> " + (list.find(9) != null));
46          System.out.println("delete 2 -> " + (list.delete(2) !=
            null));
47          list.displayList();
48          list.deleteFirst();
49          list.displayList();
50      }
```

Terminal — codeoutput/LinkList.txt

```
$ javac LinkList.java
$ java LinkList
List (first-->last): [3, 3.0] [2, 2.0] [1,
     1.0]
find 2 -> true
find 9 -> false
delete 2 -> true
List (first-->last): [3, 3.0] [1, 1.0]
List (first-->last): [1, 1.0]
```

- Three `insertFirst` calls build the list in **reverse** order: 3, 2, 1.
- `find(2)` succeeds, `find(9)` fails.
- `delete(2)` removes the interior node: 3, 1.
- `deleteFirst()` removes the head: 1.

FirstLastList

FirstLastList: fields and insertLast

codeoutput/FirstLastList.java — Java

```
1  public class FirstLastList {
2      private Link first;
3      private Link last;
4      public boolean isEmpty() { return first ==
         null; }
5      public void insertLast(int id, double dd) {
6          Link newLink = new Link(id, dd);
7          if (isEmpty()) first = newLink;
8          else           last.next = newLink;
9          last = newLink;
10     }
```

- Two fields: `first` and `last`.
- `isEmpty()` is `first == null`.
- `insertLast`: if the list is empty, `first = last = newLink`; otherwise `last.next = newLink`, then `last = newLink`.
- $O(1)$ append — no walk.

FirstLastList: deleteFirst and displayList

codeoutput/FirstLastList.java — Java

```
11      public Link deleteFirst() {
12          if (first == null) return null;
13          Link temp = first;
14          first = first.next;
15          if (first == null) last = null;
16          return temp;
17      }
18      public void displayList() {
19          System.out.print("FirstLastList: ");
20          for (Link c = first; c != null; c = c.
            next) c.displayLink();
21          System.out.println();
22      }
```

- `deleteFirst`: advance `first`; if it becomes `null`, reset `last` too. $O(1)$.
- `displayList`: the same walk as in `LinkList`, starting at `first` and stopping at `null`.
- Without the `last = null` reset, the tail would dangle — the same bug as `Bug2` in the freshman deck.

FirstLastList: the run

codeoutput/FirstLastList.java — Java

```
23      public static void main(String[] args) {
24          FirstLastList list = new FirstLastList
            ();
25          list.insertLast(1, 1.0);
26          list.insertLast(2, 2.0);
27          list.insertLast(3, 3.0);
28          list.displayList();
29          System.out.println("deleteFirst -> " +
            list.deleteFirst().iData);
30          list.displayList();
31      }
32  }
```

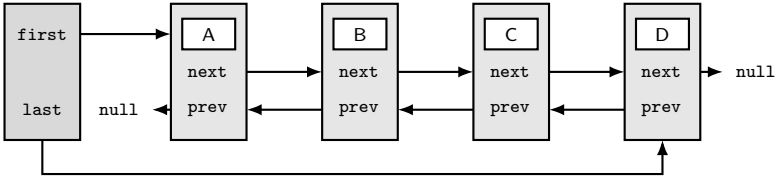
Terminal — codeoutput/FirstLastList.txt

```
$ javac FirstLastList.java
$ java FirstLastList
FirstLastList: [1, 1.0] [2, 2.0] [3, 3.0]
deleteFirst -> 1
FirstLastList: [2, 2.0] [3, 3.0]
```

- Three `insertLast` calls append in order: 1, 2, 3.
- `deleteFirst()` returns `[1, 1.0]`.
- The remaining list is 2, 3 — the order is preserved, which is exactly what a queue needs.

Doubly Linked

A node with two references



- Each node holds three fields: `data`, `next`, `prev`.
- The list keeps both ends: `first` at the head, `last` at the tail.
- Cost: one extra reference per element.
- Benefit: every operation on the two ends is $O(1)$, and iteration can go either way.

Why prev is useful

codeoutput/JavaContainers.java — Java

```java
import java.util.LinkedList;

public class JavaContainers {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.addFirst("b");
        list.addFirst("a");
        list.addLast("c");

        System.out.println("list:       " + list);
        System.out.println("getFirst(): " + list.getFirst());
        System.out.println("getLast():  " + list.getLast());

        // Peel from the tail. Each removeLast() is O(1) because of prev;
        // a singly linked list would have to walk from head every time.
        System.out.print("peel:       ");
        while (!list.isEmpty()) {
            System.out.print(list.removeLast() + " ");
        }
        System.out.println();
        System.out.println("list:       " + list);
    }
}
```

Terminal — codeoutput/JavaContainers.txt

```
$ javac JavaContainers.java
$ java JavaContainers
list:       [a, b, c]
getFirst(): a
getLast():  c
peel:       c b a
list:       []
```

- `getLast()` and `removeLast()` would need a walk on a singly linked list; here they go through `last.prev` and are $O(1)$.
- Peeling from the back (`removeLast()`) is symmetric to peeling from the front.

Uses and Wrap-up

Where a linked list wins

- FIFO or LIFO where the number of elements is not known in advance.
- Splicing two sequences (LRU cache, undo/redo).
- Music playlists (insert in the middle, skip to next).
- The bucket chain of a hash table.
- Adjacency lists of a graph.

Where it loses

- Random access by index.
- Cache-sensitive hot loops.
- Anything where you know the array size up front.

The JDK gives you both: `ArrayList` and `LinkedList`

The abstract array and the abstract linked list have concrete names in `java.util`.

`ArrayList<E>`: the array

- one resizable `Object[]`
- `get(i)`: $O(1)$
- `add(0, x)`: $O(n)$ (shift)
- `remove(0)`: $O(n)$ (shift)
- iterate: cache-friendly
- `size()`: $O(1)$ counter

`LinkedList<E>`: the chain

- one node per element (`item`, `next`, `prev`)
- `get(i)`: $O(i)$ (walk)
- `add(0, x)`: $O(1)$ (relink)
- `remove(0)`: $O(1)$ (relink)
- iterate: pointer chasing
- `size()`: $O(1)$ counter

Operation	`ArrayList`	`LinkedList`
`get(i)`	$O(1)$	$O(i)$
`add(0, x)`	$O(n)$	$O(1)$
`remove(0)`	$O(n)$	$O(1)$
iterate all	$O(n)$	$O(n)$
memory per element	4 B + ref	24 B (3 refs)

Big ideas

- A linked list is a chain of **nodes**, each with a data field and a `next` reference.
- The list object holds only `first`; `last` is optional and makes enqueue $O(1)$.
- A stack is a linked list with only `first`; a queue is a linked list with `first` and `last`.
- A doubly linked list adds `prev`, so deletion no longer needs the previous node.
- Trees, heaps and graphs are the same node-and-edge idea with an invariant. A linked list is the simplest case.
- You cannot jump; every access walks.

Check yourself

1. What does a node hold that an array element does not?
2. Why is `insertFirst` $O(1)$ but `get(k)` $O(k)$?
3. Why does a queue need `last` but a stack does not?
4. What does a doubly linked node hold that a singly linked node does not?
5. Name three of the four node-and-edge structures from the PPTX.
6. When would you pick `LinkedList` over `ArrayList` in the JDK?

The Two Exercises from Previous Years

Stutter: duplicate every element

codeoutput/Stutter.java — Java

```java
import java.util.LinkedList;

public class Stutter {
    static void stutter(LinkedList<Integer> list) {
        LinkedList<Integer> out = new LinkedList<>();
        for (int x : list) { out.add(x); out.add(x); }
        list.clear(); list.addAll(out);
    }
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        for (int i = 1; i <= 4; i++) list.add(i);
        stutter(list);
        System.out.println(list);
    }
}
```

Terminal — codeoutput/Stutter.txt

```
$ javac Stutter.java
$ java Stutter
[1, 1, 2, 2, 3, 3, 4, 4]
```

- Build a new list with each element twice, then swap it in.
- `[1, 2, 3, 4]` $\longrightarrow$ `[1, 1, 2, 2, 3, 3, 4, 4]`.
- One pass, $O(n)$ time, $O(n)$ extra space.

Mirror: append the reverse of itself

codeoutput/Mirror.java — Java

```java
import java.util.LinkedList;

public class Mirror {
    static void mirror(LinkedList<Integer> list) {
        LinkedList<Integer> copy = new LinkedList<>(list);
        java.util.Collections.reverse(copy);
        list.addAll(copy);
    }
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        for (int i = 1; i <= 4; i++) list.add(i);
        mirror(list);
        System.out.println(list);
    }
}
```

Terminal — codeoutput/Mirror.txt

```
$ javac Mirror.java
$ java Mirror
[1, 2, 3, 4, 4, 3, 2, 1]
```

- Copy the list, reverse the copy, append it.
- `[1, 2, 3, 4]` → `[1, 2, 3, 4, 4, 3, 2, 1]`.
- Symmetric input is a palindrome; the deque version needs no copy.

Classic Problems

Reverse in place

codeoutput/ReverseList.java — Java

```java
public class ReverseList {
    static class Node { final int item; Node next;
        Node(int item, Node next) { this.item = item; this.next = next; } }
    static Node reverse(Node head) {
        Node prev = null, cur = head;
        while (cur != null) { Node n = cur.next; cur.next = prev; prev = cur; cur = n; }
        return prev;
    }
    public static void main(String[] args) {
        Node head = new Node(1, new Node(2, new Node(3, new Node(4, null))));
        head = reverse(head);
        for (Node p = head; p != null; p = p.next) System.out.print(p.item + " ");
        System.out.println();
    }
}
```

Terminal — codeoutput/ReverseList.txt

```
$ javac ReverseList.java
$ java ReverseList
4 3 2 1
```

- Three pointers, one pass: `prev`, `cur`, `next`.
- Redirect each `next` backwards, then advance.
- $O(n)$ time, $O(1)$ space, no new nodes.

Palindrome check

codeoutput/Palindrome.java **Java**

```java
import java.util.LinkedList;

public class Palindrome {
    static boolean isPalindrome(LinkedList<Character> list) {
        int i = 0, j = list.size() - 1;
        while (i < j) if (!list.get(i++).equals(list.get(j--))) return false;
        return true;
    }
    public static void main(String[] args) {
        LinkedList<Character> a = new LinkedList<>(java.util.List.of('r','a','c','e','c','a','r'));
        LinkedList<Character> b = new LinkedList<>(java.util.List.of('a','b','c'));
        System.out.println(a + ": " + isPalindrome(a));
        System.out.println(b + ": " + isPalindrome(b));
    }
}
```

Terminal codeoutput/Palindrome.txt

```
$ javac Palindrome.java
$ java Palindrome
[r, a, c, e, c, a, r]: true
[a, b, c]: false
```

- Two indices, one from each end, meet in the middle.
- `r a c e c a r` → `true`; `a b c` → `false`.
- On a linked list without `get(i)`, the same idea needs a stack or a recursion.

References

1. Robert Lafore, *Data Structures & Algorithms in Java*, 2nd ed., Sams, 2002.
   The `Link` / `LinkList` / `FirstLastList` style reused in the freshman deck.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
   Chapter 10: elementary data structures, linked lists, stacks, queues.
3. Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.
   Section 1.3: linked lists; Section 3.1: symbol tables built on them.
4. *8 Basic Data Structures Every Programmer Should Know*, GoDaddy Resources.
   `https://www.godaddy.com/resources/in/web-pro-in/8-basic-data-structures-every-programmer-should-know`
   The source for the node-and-edge survey (list, tree, heap, graph) on slides 3–7 of the deck.
5. Oracle, *Java SE 21 API Specification*.
   `java.util.LinkedList`, `java.util.List`, `java.util.Deque`.
6. Oracle, *The Java Tutorials: Trail: Collections*.

Questions?